El contenedor `map`

Un diccionario ordenado por clave

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Octubre 2026

Objetivos de aprendizaje I

Al finalizar esta sesión, el estudiante podrá

- Usar `map<K, V>` para insertar con `[]`, consultar con `count` y `find`, eliminar con `erase` y recorrer las claves de menor a mayor con un iterador (OA3).
- Seguir a mano el contenido de un `map` tras una secuencia de operaciones, incluida la inserción que hace `[]` cuando solo se quería consultar (OA2).
- Dar el costo de cada operación del `map` y compararlo con el de la tabla hash con encadenamiento (OA1).
- Escoger entre la tabla hash y el `map` según lo que pide un problema, y resolver con ellos problemas de juez (OA5).

Objetivos de aprendizaje II

Al finalizar esta sesión, el estudiante podrá también

- Usar `unordered_map<K, V>`, la tabla hash de la biblioteca, explicar su factor de carga y su rehash, y escogerlo frente al `map` cuando el orden no hace falta (OA1, OA3).
- Comparar funciones hash (división, multiplicación, polinomio de Horner y `std::hash`) por cómo reparten y lo que cuestan (OA1).
- Usar un `struct` como clave: un orden para el `map`, y un hash con su igualdad para el `unordered_map` (OA3).

Contenido I

1 Repaso: la tabla hash con encadenamiento

2 Un problema donde importa el orden

3 El contenedor `map`

4 Lo que cuesta

5 El contenedor `unordered_map`

6 Claves compuestas en `map` y `unordered_map`

7 Para el juez

8 Ejercicios

Contenido I

1 Repaso: la tabla hash con encadenamiento

2 Un problema donde importa el orden

3 El contenedor `map`

4 Lo que cuesta

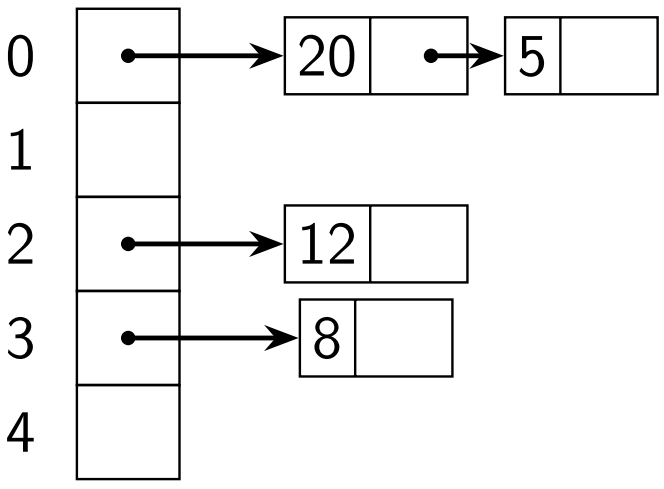
5 El contenedor `unordered_map`

6 Claves compuestas en `map` y `unordered_map`

7 Para el juez

8 Ejercicios

Lo que quedó de la tabla hash I



- $h(k) = k$ mód m lleva cada clave a una de m casillas; cada casilla es la cabeza de una lista enlazada.
- Insertar al frente cuesta $\Theta(1)$. Buscar y eliminar recorren una lista: $\Theta(1 + \alpha)$ en promedio, con $\alpha = n/m$, y $\Theta(n)$ en el peor caso.
- Espacio: $\Theta(m + n)$.

Pregunta

La tabla de la izquierda tiene $m = 5$ y guarda 5, 20, 12 y 8. ¿Cómo se imprimen esas claves de menor a mayor?

Contenido I

1 Repaso: la tabla hash con encadenamiento

2 Un problema donde importa el orden

3 El contenedor `map`

4 Lo que cuesta

5 El contenedor `unordered_map`

6 Claves compuestas en `map` y `unordered_map`

7 Para el juez

8 Ejercicios

Un inventario I

Problema

Una tienda anota cada entrada de mercancía con el código del producto y la cantidad que llegó. Al cerrar el día imprime, por cada código, el total recibido, de menor a mayor código. Los códigos llegan hasta 10^9 y en un día entran unos pocos miles de registros.

¿Entendimos el problema? I

Entradas

n y después n registros, cada uno con un código y una cantidad.

Código	Cantidad
305	4
112	2
305	1
87	6
112	3

Salidas

Una línea por código distinto con su total, en orden creciente de código.

Código	Total
87	6
112	5
305	5

¿Entendimos el problema? II

Pregunta

El 87 llegó de cuarto. ¿Por qué sale de primero? ¿Y por qué no sirve una tabla de direccionamiento directo?

¿Entendimos el problema? III

Respuesta

- La salida va por código, no por orden de llegada: 87 es el menor de los tres.
- Con códigos hasta 10^9, la tabla directa pide 10^9 casillas para unos pocos miles de códigos.
- Cada registro hace dos cosas: buscar el código y, si no estaba, agregarlo. Es el trabajo de la tabla hash.

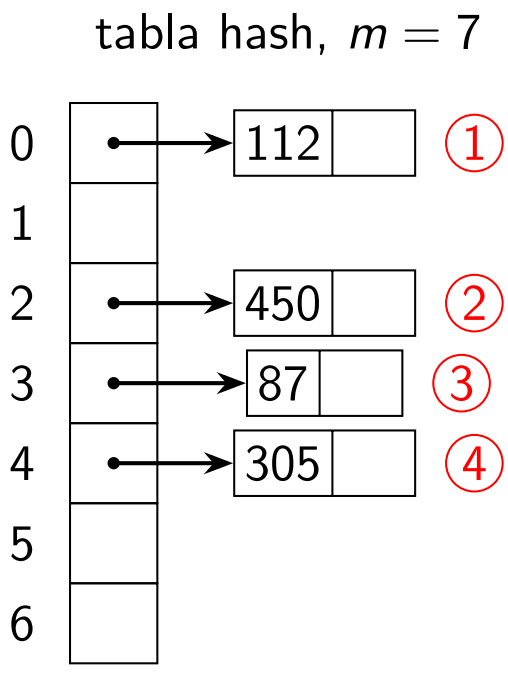
Primera idea: la tabla hash I

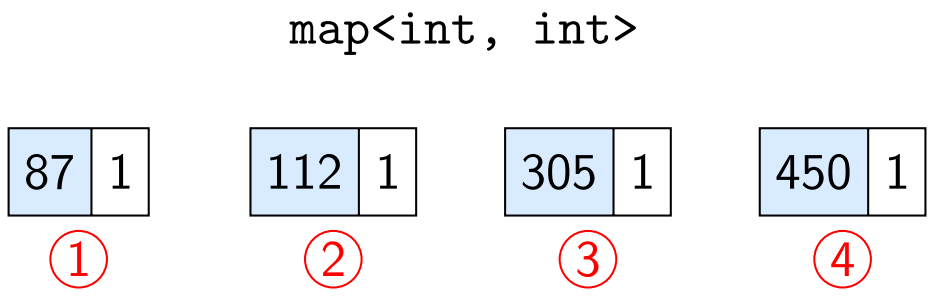
Sumar las cantidades es buscar e insertar: $\Theta(1 + \alpha)$ en promedio por registro. Falta imprimir en orden.

Pregunta

Con $m = 7$ se insertan 305, 112, 87 y 450, y después se recorre la tabla casilla por casilla con `imprimir`. ¿En qué orden salen?

Primera idea: la tabla hash II





recorrido: `112 450 87 305`

recorrido: `87 112 305 450`

305 mód 7 = 4, 112 mód 7 = 0, 87 mód 7 = 3 y 450 mód 7 = 2. Los círculos dan el orden en que sale cada clave.

Primera idea: la tabla hash III

La tabla hash ordena por casilla, no por clave

El recorrido sigue el orden de $h(k)$. Que 112 quede en la casilla 0 y 87 en la 3 no dice nada sobre cuál clave es menor: la función hash reparte las claves y en el camino deshace su orden.

Primera idea: la tabla hash IV

```
int main() {
  int claves[4] = {305, 112, 87, 450};
  TablaHash t(7);
  map<int, int> m;
  int i = 0;
  while (i < 4) {
    t.insertar(claves[i]);
    m[claves[i]] = 1;
    i = i + 1;
  }
  printf("tabla hash, casilla por casilla:\n");
  t.imprimir();
  printf("map, con su iterador:");
  map<int, int>::iterator it = m.begin();
  while (it != m.end()) {
    printf(" %d", it->first);
    ++it;
  }
  printf("\n");
  return 0;
}
```

Primera idea: la tabla hash V

Salida

```
0: 112, 1:, 2: 450, 3: 87, 4: 305, 5:, 6:
map, con su iterador: 87 112 305 450
```

La última línea viene del contenedor de esta sesión, que recorre las mismas cuatro claves de menor a mayor.

Arreglarlo al final

Se copian las n claves a un arreglo y se ordena: $\Theta(m+n)$ para recorrer y $\Theta(n \log n)$ para ordenar. Sirve si el orden se pide una vez. Si se pide tras cada registro, cada pedido vuelve a pagar el ordenamiento.

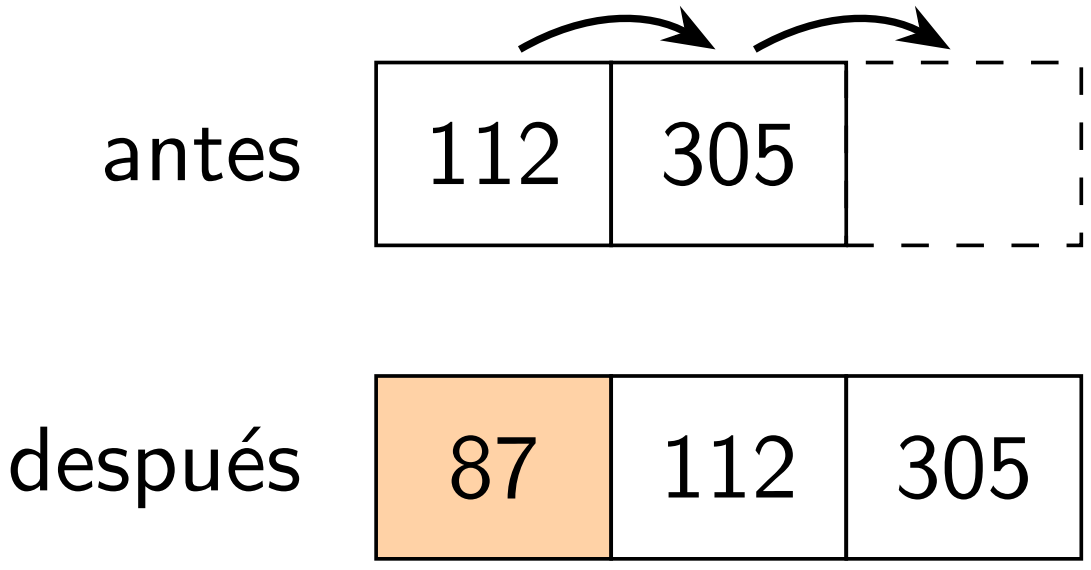
Segunda idea: un vector ordenado I

Se guardan los códigos en un vector ordenado. Buscar es una búsqueda binaria, $\Theta(\log n)$, y recorrer en orden es leer el vector.

Pregunta

El vector tiene 112 305. ¿Qué pasa al llegar el 87?

Segunda idea: un vector ordenado II



El 87 va adelante de todos: los dos códigos se corren una casilla.
Con n códigos, insertar al frente corre los n: $\Theta(n)$.

Segunda idea: un vector ordenado III

Las dos ideas, lado a lado

Ninguna de las dos da las tres cosas que pide el inventario.

	Buscar	Insertar	Recorrer en orden
Tabla hash	$\Theta(1+\alpha)$ promedio	$\Theta(1+\alpha)$ promedio	$\Theta(m+n)+\Theta(n \log n)$
Vector ordenado	$\Theta(\log n)$	$\Theta(n)$	$\Theta(n)$
Lo que se quiere	$O(\log n)$	$O(\log n)$	$\Theta(n)$

Contenido I

1 Repaso: la tabla hash con encadenamiento

2 Un problema donde importa el orden

3 El contenedor `map`

4 Lo que cuesta

5 El contenedor `unordered_map`

6 Claves compuestas en `map` y `unordered_map`

7 Para el juez

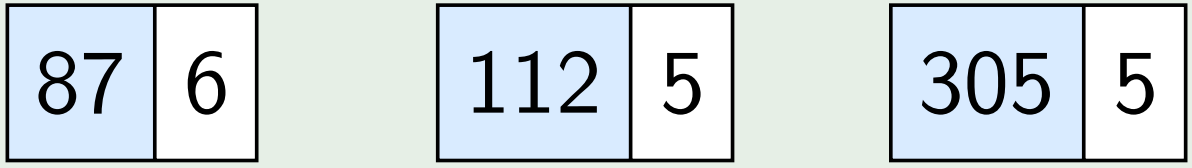
8 Ejercicios

Un diccionario ordenado I

Definición

Un `map<K, V>` guarda parejas (*clave*, *valor*) con claves distintas, siempre en orden creciente de clave. Se busca por clave, como en la tabla hash, y se recorre de la menor clave a la mayor, como un vector ordenado.

El inventario, tras los cinco registros



Las claves van en celeste y los valores en blanco. Hay tres parejas, una por código, y salen en el orden en que se imprimen.

Un diccionario ordenado II

- `K` es el tipo de la clave y `V` el del valor: `map<int, int>` para código y total, `map<string, int>` para palabra y cuenta.
- La clave tiene que poderse comparar con `<`. El `map` no pide una función hash.
- La biblioteca lo guarda en un árbol balanceado, una estructura que el curso construye más adelante [1]. De ella salen los costos de esta sesión.

```
#include <map>
using std::map;

map<int, int> inv;          // vacio: size() vale 0
```

El contrato I

Operación	Qué hace
`m[k]`	el valor de `k`; si `k` no está, inserta $(k, 0)$ y devuelve ese 0
`m[k] = v`	inserta (k, v), o cambia el valor si `k` ya estaba
`m.count(k)`	1 si `k` está, 0 si no; no inserta
`m.find(k)`	iterador a la pareja de `k`, o `m.end()` si no está
`m.erase(k)`	saca la pareja de `k`; devuelve cuántas sacó, 1 o 0
`m.size()`, `m.empty()`	cuántas parejas hay; si no hay ninguna
`m.begin()`, `m.end()`	iterador a la menor clave; iterador después de la mayor
`it->first`, `it->second`	la clave y el valor de la pareja a la que apunta `it`

El contrato II

La clave no se cambia

`it->second` se puede modificar; `it->first` no. Cambiar una clave en su sitio rompería el orden. Para cambiar una clave se saca la pareja y se inserta otra.

Insertar con []: inv[305] = 4 I

```
map<int, int> inv;        // vacio: size() = 0
inv[305] = 4;
```

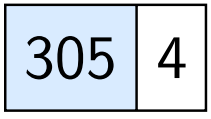
antes	(vacío)	`size() = 0`
`inv[305]`	305 | 0	305 no estaba: entra con 0
`= 4`	305 | 4	la asignación cambia el valor; `size() = 1`

`inv[305] = 4` hace dos cosas: `inv[305]` busca 305 y, al no hallarlo, lo inserta con valor 0; después la asignación pone el 4.

Insertar con []: `inv[112] = 2` I

```
inv[112] = 2;
```

antes



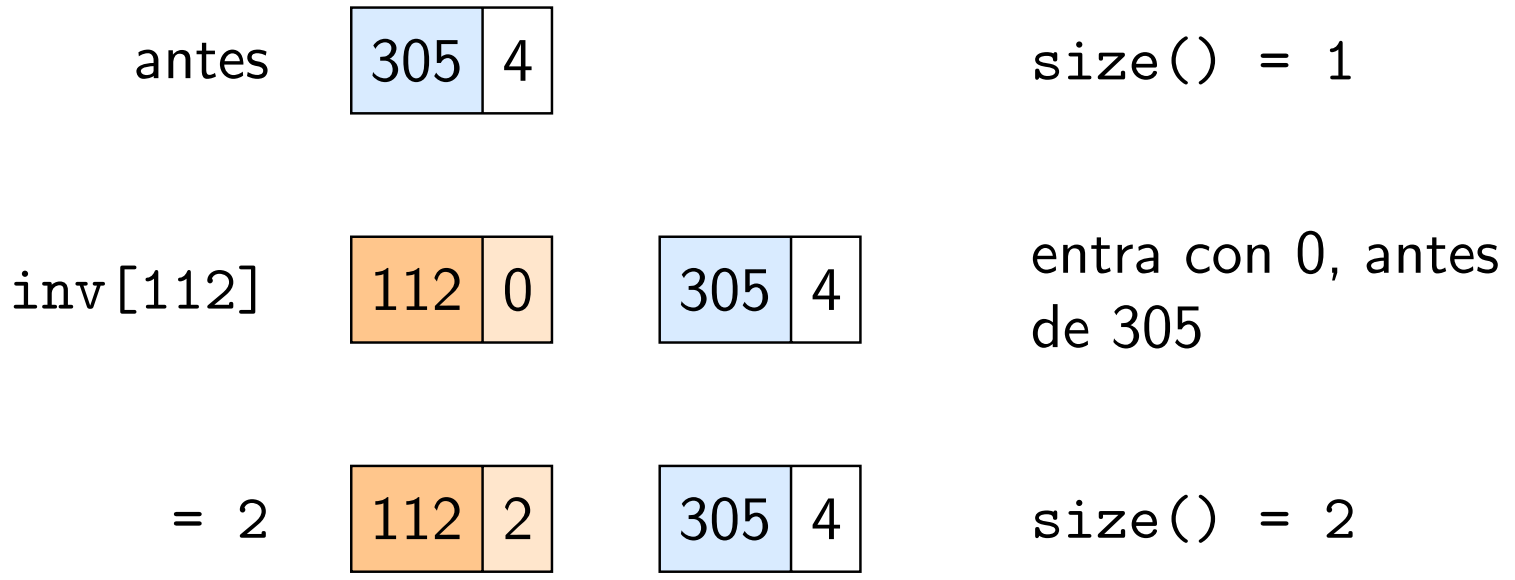
`size() = 1`

Pregunta

112 llega después de 305. ¿Dónde queda, antes o después?

Insertar con `[]`: `inv[112] = 2` II

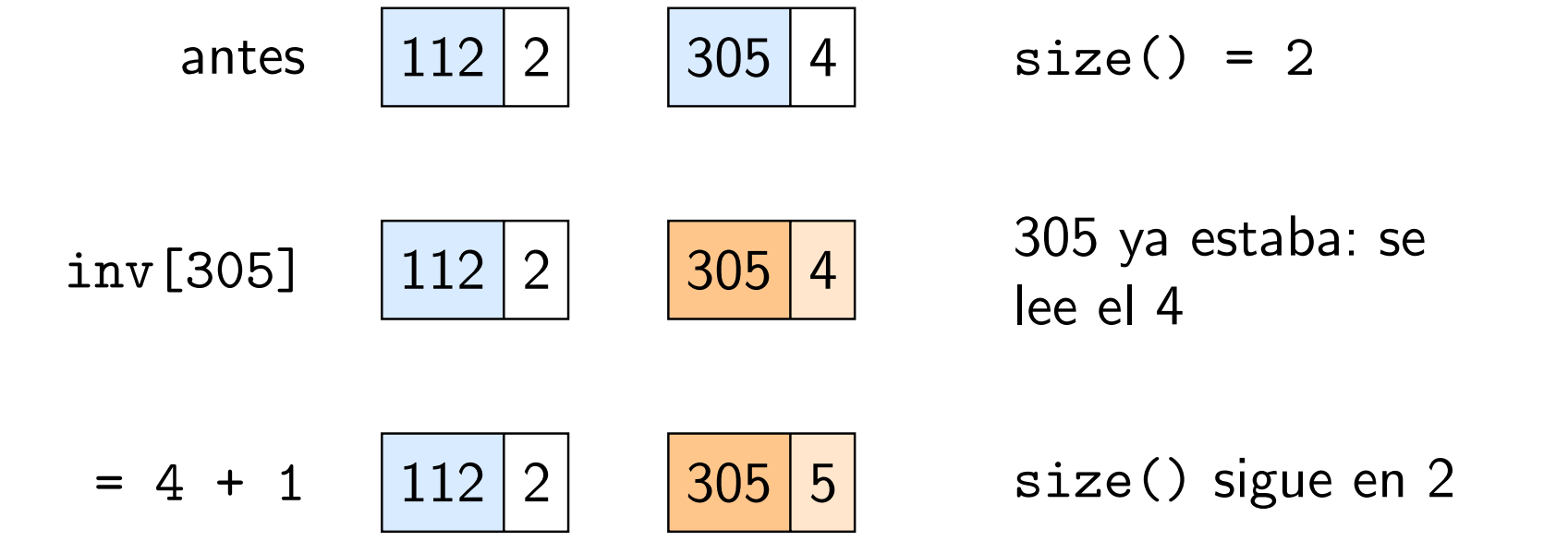
```
inv[112] = 2;
```



Antes, porque $112 < 305$. El `map` pone cada clave en su lugar al insertarla, sin correr a nadie como haría el vector.

Insertar con []: una clave que ya está I

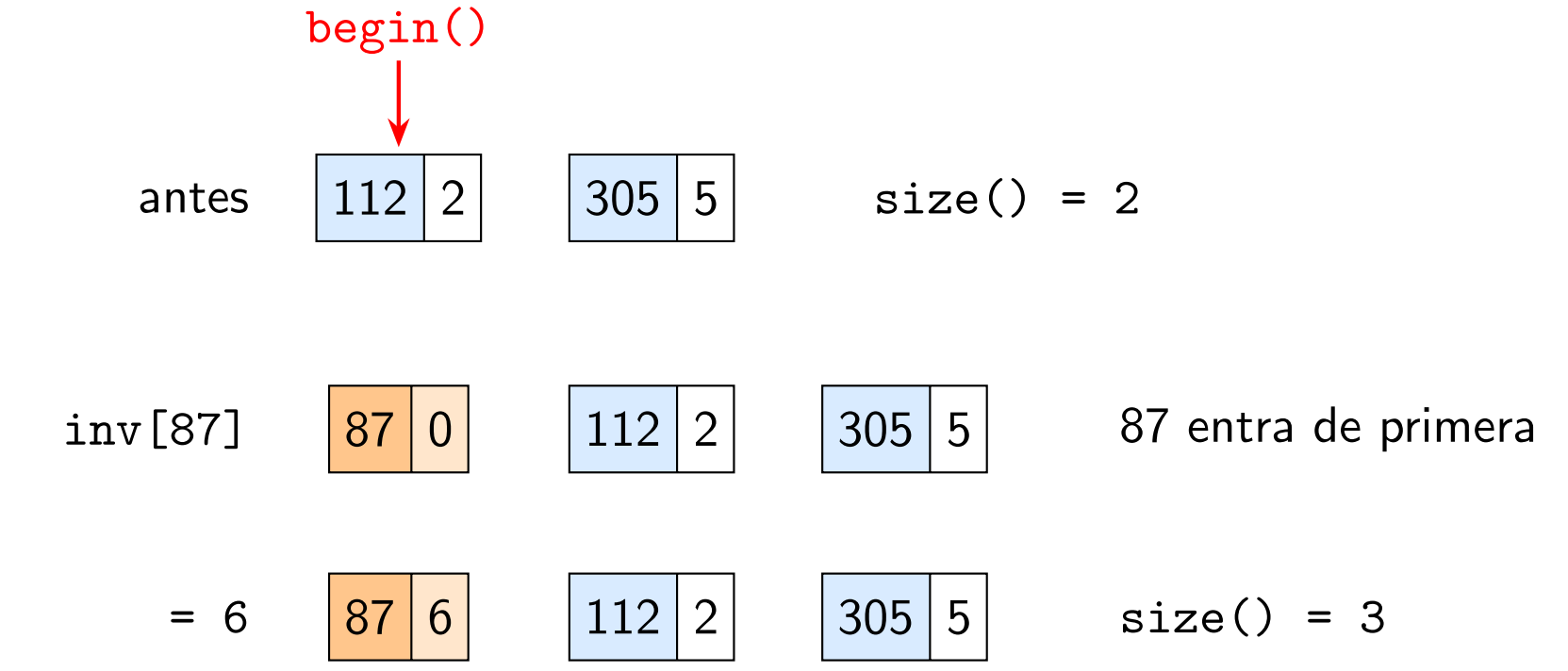
```
inv[305] = inv[305] + 1;
```



Con la clave presente, `inv[305]` no inserta: devuelve el valor que hay y la asignación lo cambia.

Insertar con []: la nueva menor clave I

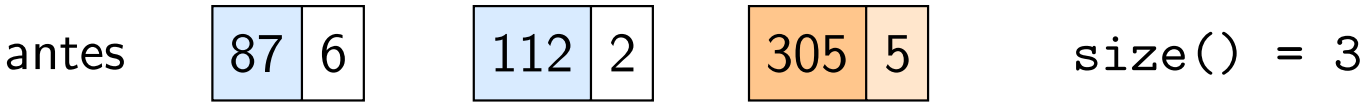
```
inv[87] = 6;
```



Antes de insertar, `begin()` apuntaba a 112. Ahora la menor clave es 87 y `begin()` apunta a ella.

Eliminar: erase I

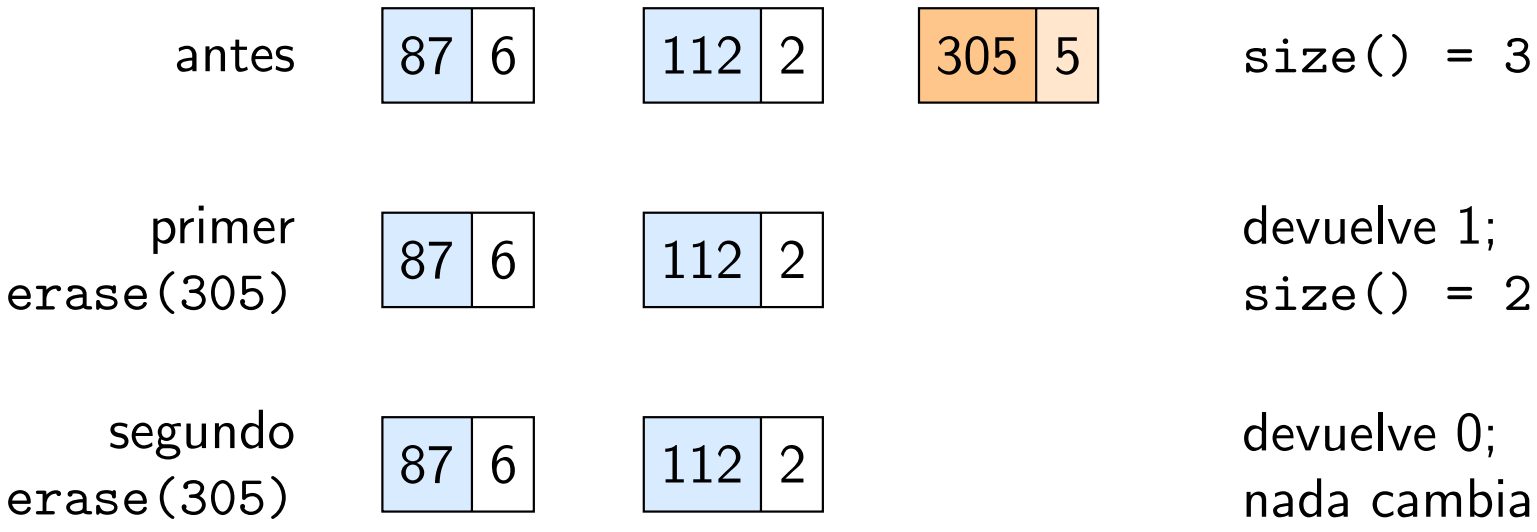
```
printf("%d\n", (int) inv.erase(305));
printf("%d\n", (int) inv.erase(305));
```



Pregunta

¿Qué imprime cada llamada, y cómo queda el `map`?

Eliminar: `erase` II



La pareja de 305 sale entera, clave y valor. Sacar una clave que no está es válido y devuelve 0, así que `erase(k)` no necesita un `count` antes.

La traza completa I

Línea	Contenido, en orden	`size()`
`map<int, int> inv;`	(vacío)	0
`inv[305] = 4;`	(305, 4)	1
`inv[112] = 2;`	(112, 2) (305, 4)	2
`inv[305] = inv[305] + 1;`	(112, 2) (305, 5)	2
`inv[87] = 6;`	(87, 6) (112, 2) (305, 5)	3
`inv.erase(305);`	(87, 6) (112, 2)	2

`size()` sube solo cuando entra una clave que no estaba, y baja solo con `erase`. Cambiar un valor no la mueve.

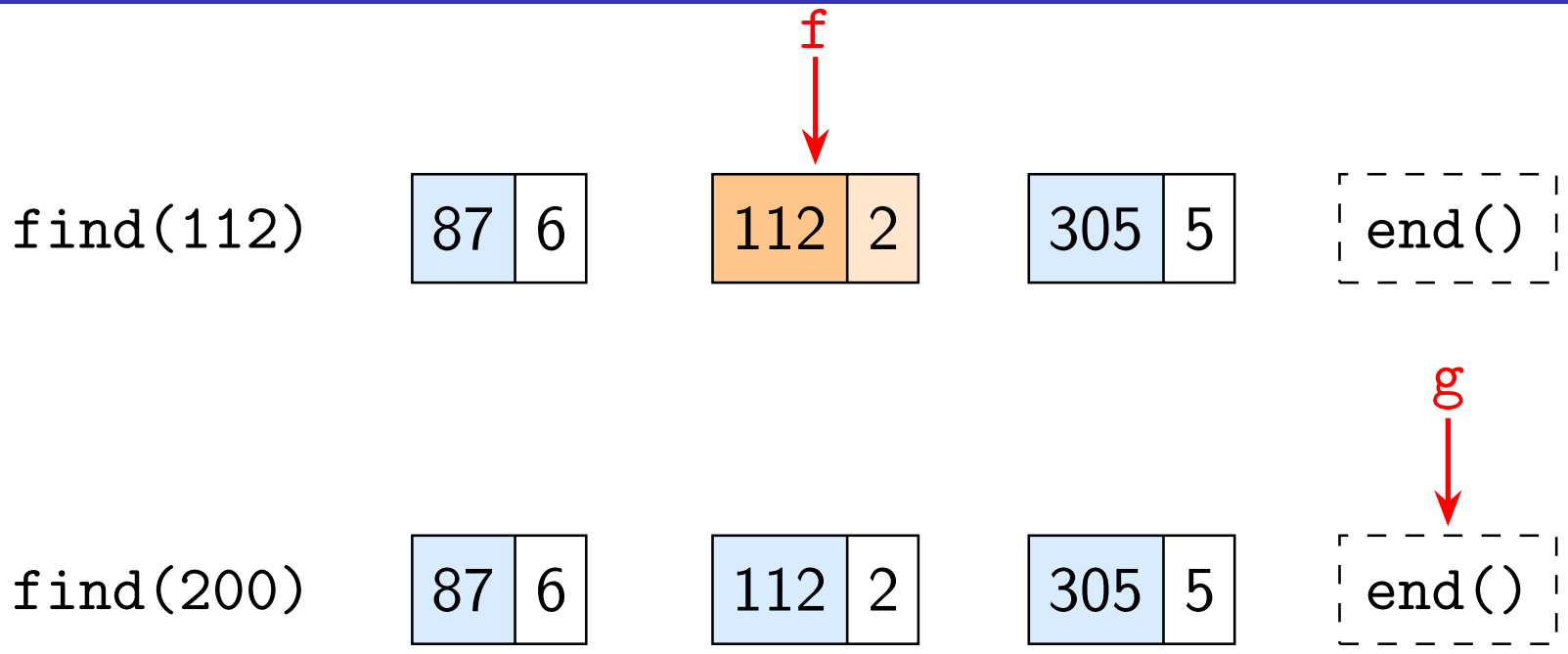
Buscar: `find` I

El inventario tiene (87, 6) (112, 2) (305, 5).

Pregunta

1. Si `f = inv.find(112)`, ¿a qué apunta `f`, y cuánto valen `f->first` y `f->second`?
2. ¿Qué devuelve `inv.find(200)`?

Buscar: `find` II



- `f` apunta a la pareja de 112: `f->first` es 112 y `f->second` es 2.
- 200 no está: `find` devuelve `inv.end()`, la posición después de la mayor clave, que no guarda ninguna pareja. Se compara con `end()` antes de leer `->second`.
- En los dos casos el `map` queda igual: `find` no inserta.

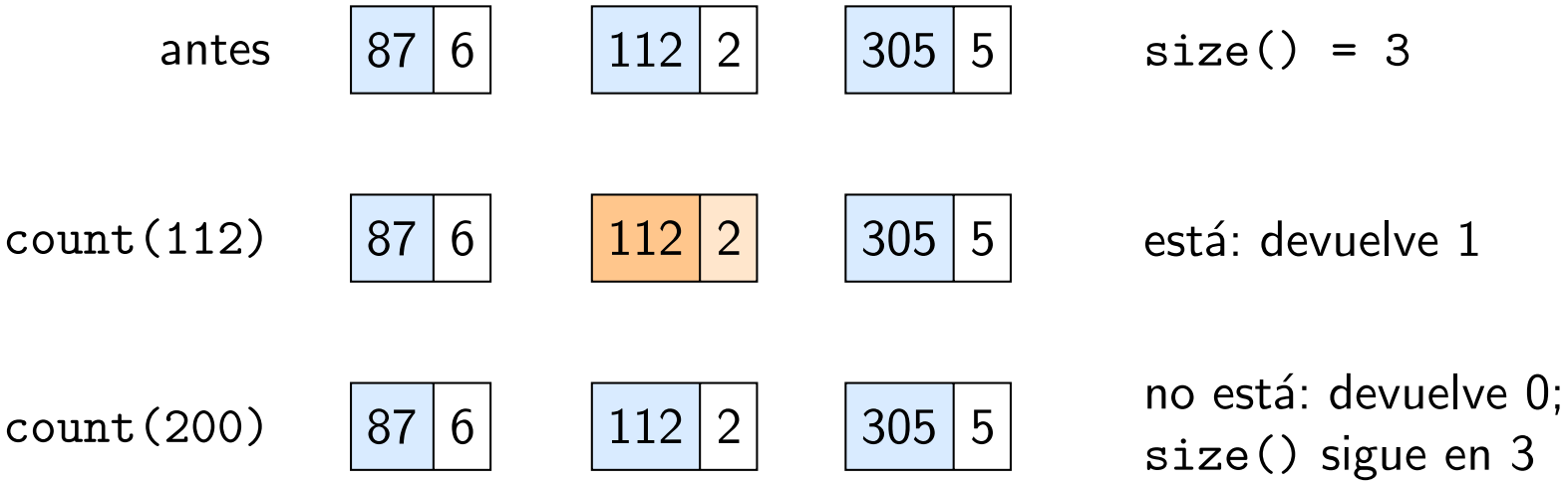
Buscar: `find` III

```
map<int, int>::iterator f = inv.find(112);
if (f != inv.end()) {
  printf("find(112): clave %d, valor %d\n", f->first, f->second);
}
printf("find(200) == end(): %d\n", (int) (inv.find(200) ==
↪  inv.end()));
```

Salida

```
find(112): clave 112, valor 2 y find(200) == end(): 1.
```

Contar: `count` I



Una clave aparece una vez o ninguna, así que `count` solo devuelve 1 o 0. Responde si está; `find` además deja en la mano la pareja, sin buscarla dos veces.

```
printf("count(112) = %d, count(200) = %d\n", (int) inv.count(112),
↪  (int) inv.count(200));
```

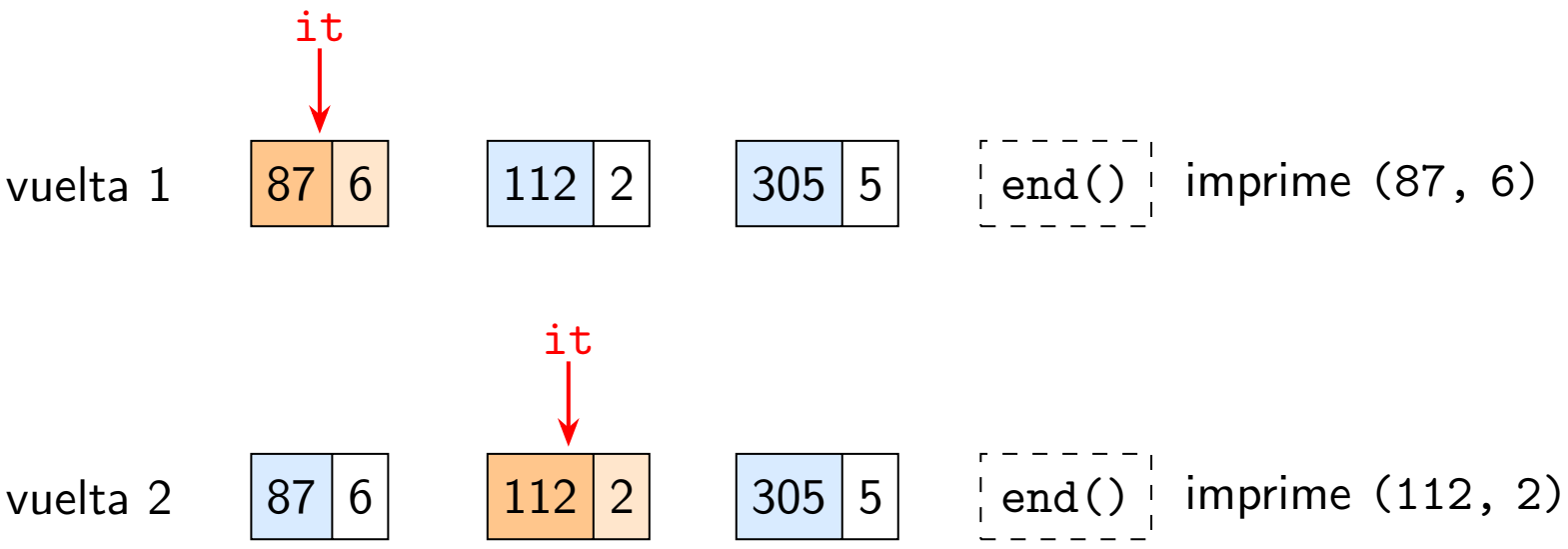
Recorrer en orden I

El recorrido es el mismo de `vector` y `list`: un iterador desde `begin()` hasta `end()`. Lo nuevo es que cada posición guarda una pareja.

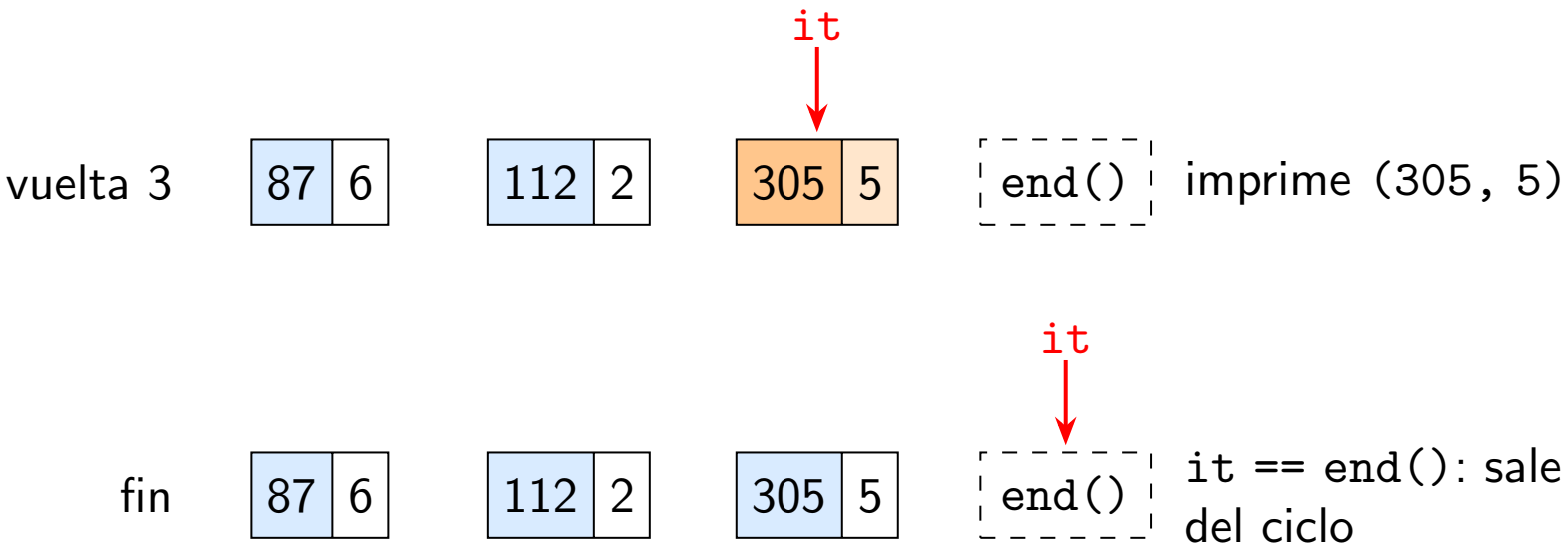
```
// recorre el map con su iterador: las claves salen de menor a mayor
void imprimir(map<int, int> &m) {
  map<int, int>::iterator it = m.begin();
  while (it != m.end()) {
    printf(" (%d, %d)", it->first, it->second);
    ++it;
  }
  printf("   size = %d\n", (int) m.size());
}
```

- `it->first` es la clave y `it->second` el valor.
- `++it` pasa a la siguiente clave mayor.

Recorrer en orden II



Recorrer en orden III



Las claves salen de menor a mayor sin ordenar nada: el orden ya está en el `map`.

Recorrer en orden IV

Un programa hace las líneas de la traza, llama a `imprimir` después de cada una y termina con las consultas de `count`, `find` y `erase`:

Salida

```
 (305, 4)   size = 1
 (112, 2) (305, 4)   size = 2
 (112, 2) (305, 5)   size = 2
 (87, 6) (112, 2) (305, 5)   size = 3
count(112) = 1, count(200) = 0
find(112): clave 112, valor 2
find(200) == end(): 1
erase(305) = 1
erase(305) = 0
 (87, 6) (112, 2)   size = 2
```

Las cuatro primeras líneas y la última son filas de la traza.

Preguntar con corchetes I

El inventario tiene (87, 6) (112, 5) (305, 5). Se consultan 200, 112 y 999 así:

```
while (i < 3) {
  int k = consultas[i];
  if (inv[k] != 0) {              // si k no estaba, esta linea lo
  ↪  inserta con 0
    printf("%d: hay %d\n", k, inv[k]);
  } else {
    printf("%d: no hay\n", k);
  }
  printf("   size = %d\n", (int) inv.size());
  i = i + 1;
}
```

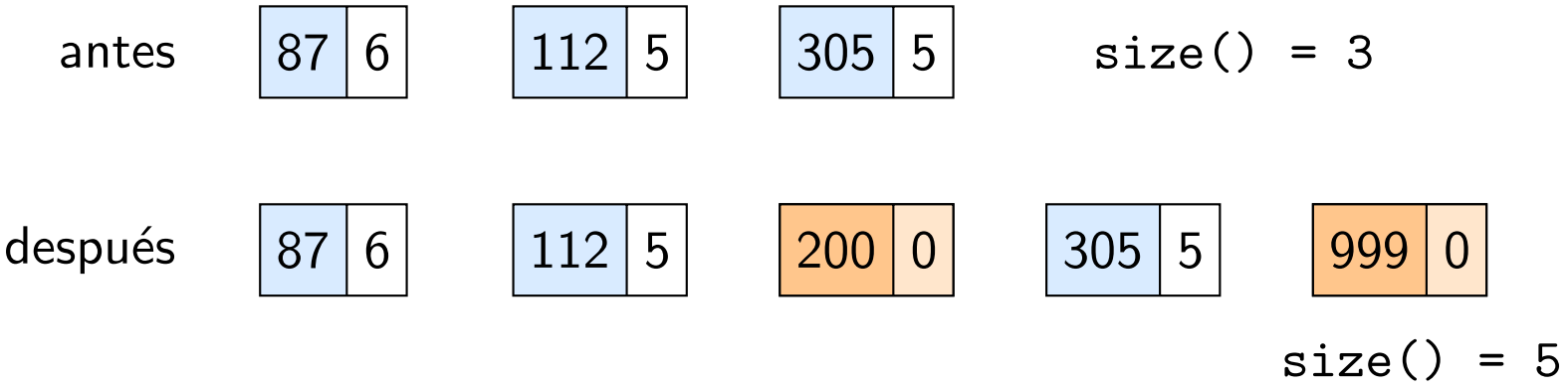
Pregunta

¿Qué imprime para cada consulta, y cuánto vale `size()` al final?

Preguntar con corchetes II

Salida

```
200: no hay, size = 4; 112: hay 5, size = 4; 999: no hay, size
= 5.
```



Las respuestas son correctas, pero entraron (200, 0) y (999, 0), dos productos que nunca llegaron. Un recorrido los imprimiría con total 0.

Preguntar con corchetes III

`m[k]` inserta

Si `k` no está, `m[k]` inserta $(k, 0)$ antes de devolver el valor. Para preguntar si una clave está se usa `count` o `find`, que no tocan el `map`.

Preguntar con corchetes IV

```
while (i < 3) {
  int k = consultas[i];
  if (otro.count(k) == 1) {
    printf("%d: hay %d\n", k, otro[k]);
  } else {
    printf("%d: no hay\n", k);
  }
  printf("   size = %d\n", (int) otro.size());
  i = i + 1;
}
```

Preguntar con corchetes V

Con `count`, las tres consultas imprimen lo mismo y `size` se queda en 3. Dentro del `if`, `otro[k]` ya no inserta nada: la clave está.

La misma inserción, a favor

En el inventario, `total[codigo] = total[codigo] + cantidad` funciona porque un código nuevo entra con 0. La inserción que estorba al consultar es la que hace funcionar el conteo.

Claves que son palabras I

Problema

Leer palabras hasta el fin de la entrada e imprimir cada una con las veces que apareció, en orden alfabético.

Pregunta

Las palabras se leen con `scanf("%s", pal)` en un `char pal[101]`. ¿Sirve `map<char *, int>`, con `pal` como clave?

Claves que son palabras II

No sirve: compara direcciones

Con `map<char *, int>` la clave es un puntero, y dos punteros se comparan por la dirección, no por las letras. Todas las palabras se leen en el mismo arreglo `pal`, así que todas son la misma clave. Con la entrada `sol mar sol luna mar sol`, el `map` termina con una sola pareja, `sol 6`: la última palabra leída con la cuenta de las seis.

La clave es un `string`

`string clave = pal;` copia las letras en un `string` propio, y dos `string` se comparan con `<` en orden alfabético, como `string("cola") < string("pila")`. Para imprimirlo con `printf` se usa `c_str()`.

Claves que son palabras III

```cpp
int main() {
  map<string, int> veces;
  char pal[101];
  while (scanf("%s", pal) == 1) {
    string clave = pal;     // copia las letras: dos string se
    ↪  comparan por su texto
    veces[clave] = veces[clave] + 1;
  }
  map<string, int>::iterator it = veces.begin();
  while (it != veces.end()) {
    printf("%s %d\n", it->first.c_str(), it->second);
    ++it;
  }
  return 0;
}
```

Claves que son palabras IV

Con `sol mar sol luna mar sol`

`luna` 1, `mar` 2, `sol` 3.

Comparar dos palabras de largo L cuesta hasta L pasos, así que cada operación con claves de texto cuesta $O(L \log n)$.

El inventario, resuelto I

```cpp
int main() {
  int n;
  scanf("%d", &n);
  map<int, int> total;
  int i = 0;
  while (i < n) {
    int codigo, cantidad;
    scanf("%d %d", &codigo, &cantidad);
    total[codigo] = total[codigo] + cantidad;    // si no estaba,
    ↪  entra con 0
    i = i + 1;
  }
  map<int, int>::iterator it = total.begin();
  while (it != total.end()) {
    printf("%d %d\n", it->first, it->second);
    ++it;
  }
  return 0;
}
```

El inventario, resuelto II

Con los cinco registros del problema

`87 6`, `112 5` y `305 5`.

- Cada registro hace una operación `[]` sobre un `map` de a lo sumo n claves: $O(\log n)$. Con n registros, $O(n \log n)$.
- El recorrido final visita las d parejas una vez: $\Theta(d)$, con $d \leq n$ códigos distintos.
- Espacio: $\Theta(d)$, una pareja por código distinto.

Contenido I

1 Repaso: la tabla hash con encadenamiento

2 Un problema donde importa el orden

3 El contenedor `map`

4 Lo que cuesta

5 El contenedor `unordered_map`

6 Claves compuestas en `map` y `unordered_map`

7 Para el juez

8 Ejercicios

¿De dónde sale $\log n$? I

Pregunta

Un `map` tiene un millón de claves. ¿Cuántas comparaciones hace, más o menos, `find`?

¿De dónde sale $\log n$? II

Como la búsqueda binaria sobre un arreglo ordenado: cada comparación descarta cerca de la mitad de las claves que quedan.

Comparaciones	0	1	2	3	$\cdots$	20
Claves que quedan	10^6	$5 \cdot 10^5$	$2{,}5 \cdot 10^5$	$1{,}25 \cdot 10^5$	$\cdots$	1

$\log_2 10^6 \approx 19{,}9$: unas veinte comparaciones, frente al millón que haría una lista enlazada.

- El vector ordenado también busca así, pero insertar le cuesta $\Theta(n)$ por correr los elementos.
- El árbol balanceado del `map` mantiene buscar, insertar y eliminar en $O(\log n)$, también en el peor caso [1].

Lo que cobra el `map` I

Operación	Tiempo	Por qué
`m[k]`, `m[k] = v`	$O(\log n)$	se busca `k`; si no está, entra en su lugar sin correr a nadie
`count(k)`, `find(k)`	$O(\log n)$	la misma búsqueda, sin insertar
`erase(k)`	$O(\log n)$	se busca `k` y se saca su pareja
`size()`, `empty()`	$\Theta(1)$	el `map` lleva la cuenta
`begin()`	$\Theta(1)$	la biblioteca tiene a mano la menor clave
recorrido completo	$\Theta(n)$	cada pareja se visita una vez
destructor	$\Theta(n)$	se libera un nodo por pareja

Espacio

$\Theta(n)$: un nodo por pareja, con la clave, el valor y los punteros de la estructura. Ninguna operación pide memoria aparte que crezca con n. Con claves de texto de largo L, cada comparación cuesta hasta L y las operaciones pasan a $O(L \log n)$.

Tabla hash frente a `map` I

	Tabla hash, m proporcional a n	`map`
Buscar, insertar, eliminar	$\Theta(1+\alpha)$ promedio; $\Theta(n)$ peor	$O(\log n)$, también en el peor caso
Recorrer en orden de clave	$\Theta(m+n)$ y ordenar, $\Theta(n \log n)$	$\Theta(n)$
La menor clave	$\Theta(m+n)$, revisando todo	$\Theta(1)$ con `begin()`
Espacio	$\Theta(m+n)$	$\Theta(n)$
Lo que pide de la clave	una función hash h	comparar con `<`

Pregunta

En promedio, $\Theta(1)$ le gana a $O(\log n)$. ¿Por cuánto, con $n = 10^5$ y $n = 10^6$?

Tabla hash frente a map II

```
void medirMap(int n) {
  clock_t ini = clock();
  map<int, int> mp;
  int i = 0;
  while (i < n) {
    mp[(int) ((long long) i * 7919 % P)] = i;
    i = i + 1;
  }
  int hallados = 0;
  i = 0;
  while (i < n) {
    if (mp.count((int) ((long long) i * 7919 % P)) == 1) {
      hallados = hallados + 1;
    }
    i = i + 1;
  }
  double s = (double) (clock() - ini) / CLOCKS_PER_SEC;
  printf("map,  n = %d: %.3f s (%d hallados)\n", n, s, hallados);
}
```

Tabla hash frente a `map` III

Un programa inserta n claves distintas y después las busca todas, primero en la tabla hash, con m primo cercano a n, y después en un `map`. Esta es la parte del `map`; `medirHash` es igual, con `TablaHash t(m)`, `insertar` y `buscar`.

Tabla hash frente a `map` IV

Las cifras de las dos están en la medición con el `unordered_map`, al final de la sección siguiente.

- El `map` entrega a cambio el orden: el recorrido sale de menor a mayor sin pagar un ordenamiento.

¿Cuál escoger? I

Tabla hash

Cuando solo se pregunta si una clave está y qué valor tiene, con muchas consultas, y el orden de las claves no aparece en la salida.

`map`

Cuando la salida va en orden de clave, cuando se pide la menor o la mayor clave en cualquier momento, o cuando no hay a mano una buena función hash para la clave y sí una comparación $<$.

El peor caso

La tabla hash depende de que h reparta bien: con una mala m o con claves hechas a su medida, buscar llega a $\Theta(n)$. El `map` se queda en $O(\log n)$ con cualquier orden de llegada de las claves.

Contenido I

1 Repaso: la tabla hash con encadenamiento

2 Un problema donde importa el orden

3 El contenedor `map`

4 Lo que cuesta

5 El contenedor `unordered_map`

6 Claves compuestas en `map` y `unordered_map`

7 Para el juez

8 Ejercicios

¿Trae la biblioteca una tabla hash? I

Pregunta

La tabla hash con encadenamiento busca en $O(1)$ en promedio y el `map` en $O(\log n)$. Cuando el orden no hace falta, ¿hay que escribir la tabla a mano, o la biblioteca trae una?

¿Trae la biblioteca una tabla hash? II

Definición

Un `unordered_map<K, V>` guarda parejas (*clave*, *valor*) con claves distintas en una tabla hash con encadenamiento, la de CLRS, capítulo 11, §11.2 [1, 3]. Busca por clave como el `map`, pero las claves no quedan en orden.

```cpp
#include <unordered_map>
using std::unordered_map;

unordered_map<int, int> inv;   // vacio: size() vale 0
```

- La clave necesita una función hash y una comparación con ==. Para `int` y `string` la biblioteca ya trae las dos.

La misma interfaz que map I

El inventario de la sección del map, con unordered_map. Solo cambia el tipo:

```
int main() {
  unordered_map<int, int> inv;
  inv[305] = 4;
  inv[112] = 2;
  inv[305] = inv[305] + 1;
  inv[87] = 6;
  imprimir(inv);
  printf("count(112) = %d, count(200) = %d\n", (int) inv.count(112),
  ↪  (int) inv.count(200));
  unordered_map<int, int>::iterator f = inv.find(112);
  if (f != inv.end()) {
    printf("find(112): clave %d, valor %d\n", f->first, f->second);
  }
  printf("erase(305) = %d\n", (int) inv.erase(305));
  imprimir(inv);
  return 0;
}
```

La misma interfaz que map II

`imprimir` recorre con un iterador, como en el `map`:

```
void imprimir(unordered_map<int, int> &m) {
  unordered_map<int, int>::iterator it = m.begin();
  while (it != m.end()) {
    printf(" (%d, %d)", it->first, it->second);
    ++it;
  }
  printf("   size = %d\n", (int) m.size());
}
```

Salida

```
 (87, 6) (112, 2) (305, 5)   size = 3
count(112) = 1, count(200) = 0
find(112): clave 112, valor 2
erase(305) = 1
 (87, 6) (112, 2)   size = 2
```

`[]`, `find`, `count`, `erase`, `size`, `it->first` e `it->second` hacen lo mismo que en el `map`, y `m[k]` también inserta $(k, 0)$ si `k` no estaba.

Repaso: lo que pide una función hash I

Una buena función hash

- Es determinista: la misma clave da siempre la misma cubeta.
- Reparte parejo: claves distintas caen en cubetas distintas tan a menudo como se pueda.
- Es rápida: $\Theta(1)$ para un entero, $\Theta(L)$ para una cadena de L caracteres.

Método de la división (CLRS, §11.3)

$h(k) = k$ mód m, con m primo y lejos de una potencia de 2:
k mód 2^p son solo los últimos p bits de k.

Repaso: lo que pide una función hash II

Los múltiplos de 8, de 8 a 64, con `hash_division.cpp`:

k	8	16	24	32	40	48	56	64
k mód 16	8	0	8	0	8	0	8	0
k mód 17	8	16	7	15	6	14	5	13

Con $m = 16$ las ocho claves caen en dos cubetas; con el primo 17, cada una queda en la suya.

Cadenas

Sumar los códigos de los caracteres no ve el orden: `amor`, `roma` y `mora` suman lo mismo y chocan. El polinomio de Horner, $h \leftarrow (h \cdot b + c)$ mód m, hace pesar a cada carácter según su posición.

Horner, carácter por carácter I

```cpp
// Horner en base 31, reduciendo modulo m en cada paso
int hashPolinomio(string s, int m) {
  int h = 0;
  int i = 0;
  while (i < (int) s.size()) {
    h = (h * 31 + s[i]) % m;
    i = i + 1;
  }
  return h;
}
```

Con $b = 31$ y $m = 13$, sobre `amor`:

Paso	Carácter	Código	$h \leftarrow (h \cdot 31 + c)$ mód 13
0			$h = 0$
1	a	97	$(0 \cdot 31 + 97)$ mód $13 = 6$
2	m	109	$(6 \cdot 31 + 109)$ mód $13 = 295$ mód $13 = 9$
3	o	111	$(9 \cdot 31 + 111)$ mód $13 = 390$ mód $13 = 0$
4	r	114	$(0 \cdot 31 + 114)$ mód $13 = 10$

Horner, carácter por carácter II

Salida de `hash_cadenas.cpp`, con $m = 13$

```
amor: suma 2, polinomio 10
roma: suma 2, polinomio 0
mora: suma 2, polinomio 11
```

- Con la suma, las tres palabras van a la cubeta 2. Con el polinomio, cada una va a una cubeta distinta.
- `hashPolinomio` da una vuelta por carácter: $\Theta(L)$ en tiempo y $\Theta(1)$ de espacio aparte.

El método de la multiplicación I

Pregunta

Con $m = 16$ la división deja los múltiplos de 8 en dos cubetas. ¿Hay una función hash que funcione bien también con m potencia de 2?

El método de la multiplicación II

Definición (CLRS, §11.3)

Se fija una constante $0 < A < 1$. Se multiplica k por A, se toma la parte fraccionaria y se multiplica por m:

$$h(k) = \lfloor m\,(kA \bmod 1)\rfloor.$$

Aquí $A = (\sqrt{5} - 1)/2 \approx 0{,}6180339887$. El valor de m ya no importa tanto, y puede ser una potencia de 2.

```
const double A = 0.6180339887;   // (raiz de 5 - 1) / 2

int hashMult(int k, int m) {
  double t = k * A;
  double frac = t - (long long) t;   // parte fraccionaria de k * A
  return (int) (m * frac);
}
```

El método de la multiplicación III

Un ejemplo con $k = 123456$ y $m = 10\,000$

$kA = 76300{,}0041\ldots$; la parte fraccionaria es $0{,}0041\ldots$;
$10\,000 \cdot 0{,}0041\ldots = 41{,}1\ldots$, y $h(k) = 41$.

Los múltiplos de 8, de 8 a 64, con $m = 16$:

k	8	16	24	32	40	48	56	64
$h(k)$, multiplicación	15	14	13	12	11	10	9	8
k mód 16	8	0	8	0	8	0	8	0

`hashMult` hace una multiplicación y una resta: $\Theta(1)$.

std::hash en g++ I

unordered_map calcula el valor hash con std::hash<K> y lo reduce a una cubeta con el residuo entre bucket_count().

Salida de std_hash.cpp

```
hash<int>(305) = 305, 305 mod 13 = 6, bucket = 6
hash<int>(112) = 112, 112 mod 13 = 8, bucket = 8
hash<int>(87) = 87, 87 mod 13 = 9, bucket = 9
hash<int>(450) = 450, 450 mod 13 = 8, bucket = 8
hash<string>(amor) = 4670080542026640O5
hash<string>(roma) = 74373659541729615 82
```

- std::hash<int> devuelve el mismo entero, y bucket(k) es k mód m: el método de la división, con m primo. $\Theta(1)$.
- std::hash<string> mezcla todos los caracteres en un número de 64 bits; amor y roma dan valores distintos. $\Theta(L)$.

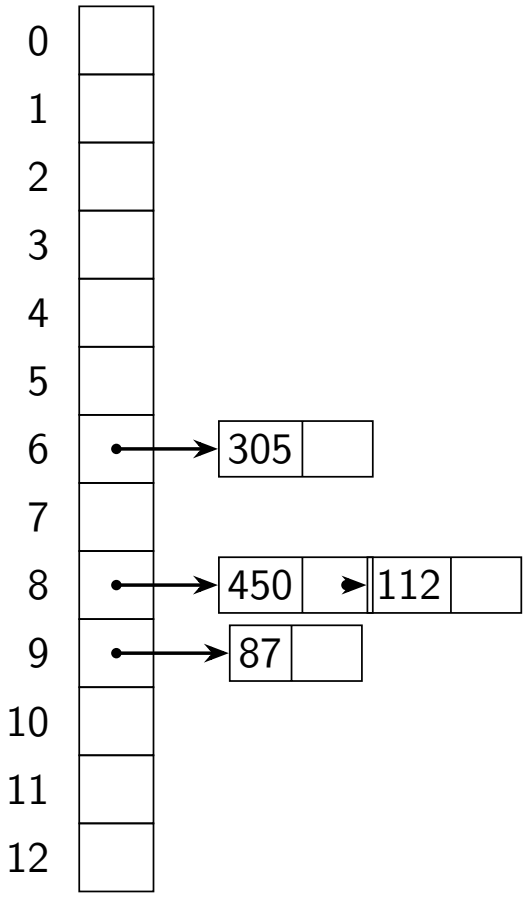
Por dentro: cubetas con listas I

La biblioteca llama *cubetas* (*buckets*) a las casillas de la tabla. `bucket_count()` dice cuántas hay y `bucket(k)` en cuál está `k`.

Pregunta

Se insertan 305, 112, 87 y 450 en un `unordered_map<int, int>`. En g++, la función hash de un `int` devuelve el mismo número y la tabla arranca con 13 cubetas. ¿En qué cubeta cae cada clave?

Por dentro: cubetas con listas II



305 mód 13 = 6, 112 mód 13 = 8, 87 mód 13 = 9 y 450 mód 13 = 8: el 112 y el 450 chocan y comparten la lista de la cubeta 8. Es la tabla de `tabla_hash.h`: un arreglo de cubetas, cada una con su lista enlazada.

Salida de `cubetas.cpp`

```
bucket_count() = 13
load_factor() = 0.308
bucket(305) = 6
bucket(112) = 8
bucket(87) = 9
bucket(450) = 8
```

El recorrido no sale en orden I

Antes de correrlo

Un programa inserta 305, 112, 87 y 450 en un `map` y en un `unordered_map`, y recorre los dos con su iterador. Escriban las dos líneas que imprime.

El recorrido no sale en orden II

```
int main() {
  int claves[4] = {305, 112, 87, 450};
  map<int, int> m;
  unordered_map<int, int> u;
  int i = 0;
  while (i < 4) {
    m[claves[i]] = 1;
    u[claves[i]] = 1;
    i = i + 1;
  }
  imprimirMap(m);
  imprimirHash(u);
  return 0;
}
```

El recorrido no sale en orden III

```
void imprimirMap(map<int, int> &m) {
  printf("map:");
  map<int, int>::iterator it = m.begin();
  while (it != m.end()) {
    printf(" %d", it->first);
    ++it;
  }
  printf("\n");
}
```

```
void imprimirHash(unordered_map<int, int> &u) {
  printf("unordered_map:");
  unordered_map<int, int>::iterator it = u.begin();
  while (it != u.end()) {
    printf(" %d", it->first);
    ++it;
  }
  printf("\n");
}
```

El recorrido no sale en orden IV

map

```
87 112 305 450
```

unordered_map

```
87 450 112 305
```

- El `map` sale de menor a mayor.
- El `unordered_map` sale como la biblioteca enlaza sus cubetas: las cubetas 9, 8, 8 y 6, que no es el orden de las claves ni el de las cubetas de menor a mayor.
- Un programa que imprima con el `unordered_map` y espere la salida en orden falla en el juez.

Factor de carga y rehash I

Pregunta

Con 13 cubetas fijas, ¿qué le pasa a la búsqueda cuando ya hay mil claves?

Factor de carga y rehash II

Definición

El factor de carga es $\alpha = n/m$, con n claves y m cubetas; lo da `load_factor()`. `max_load_factor()` vale 1: si una inserción deja $\alpha > 1$, la tabla pide más cubetas, cerca del doble y en número primo, y vuelve a colgar todas las claves en sus cubetas nuevas. Eso es un *rehash*.

Factor de carga y rehash III

n al insertar	`bucket_count()`
0	1
1	13
14	29
30	59
60	127
128	257
258	541
542	1 109

Las filas son las inserciones de `rehash.cpp`, de una en una, en las que `bucket_count()` cambió. Entre fila y fila, α sube hasta 1 sin que la tabla crezca.

Factor de carga y rehash IV

- El rehash recorre las n claves y las cuelga otra vez: $\Theta(n)$ en esa inserción.
- Después de crecer de m a cerca de $2m$ cubetas, hacen falta unas m inserciones más para volver a $\alpha = 1$. El costo $\Theta(m)$ del rehash se reparte entre esas m inserciones, y en promedio cada inserción sigue costando $O(1)$.
- Como $\alpha \leq 1$, cada lista tiene en promedio a lo sumo una clave, y buscar cuesta $O(1)$ en promedio para cualquier n.

Traza de un rehash I

Entran 5, 18, 31, 1, 2, 3, 4, 6, 7, 8, 9, 10, 11 y 12, en ese orden.
Con 13 cubetas, 5, 18 y 31 caen en la cubeta 5.

Entra	n	m	α	`bucket(5)`	`bucket(18)`	`bucket(31)`
5	1	13	0,08	5	5	5
18	2	13	0,15	5	5	5
31	3	13	0,23	5	5	5
1	4	13	0,31	5	5	5
2	5	13	0,38	5	5	5
3	6	13	0,46	5	5	5
4	7	13	0,54	5	5	5
6	8	13	0,62	5	5	5
7	9	13	0,69	5	5	5
8	10	13	0,77	5	5	5
9	11	13	0,85	5	5	5
10	12	13	0,92	5	5	5
11	13	13	1,00	5	5	5
12	14	29	0,48	5	18	2

Traza de un rehash II

- Con el 11, α llega a $13/13 = 1$: no pasa de 1 y la tabla no crece.
- El 12 dejaría $\alpha = 14/13 > 1$. La tabla pasa a 29 cubetas, reubica las 13 claves que tenía y cuelga el 12: $\alpha = 14/29 \approx 0{,}48$.
- La cubeta 5 tenía una lista de tres claves. Con 29 cubetas, 5 mód 29 = 5, 18 mód 29 = 18 y 31 mód 29 = 2: cada una queda sola.
- Las trece inserciones anteriores costaron $\Theta(1)$ cada una; la del 12 pagó las 14.

Lo que cobra cada uno I

Operación	`map`	`unordered_map`	Por qué
Buscar	$O(\log n)$	$O(1)$ promedio; $\Theta(n)$ peor	el `map` baja por un árbol balanceado; el otro calcula la cubeta y recorre su lista, de largo $\alpha \leq 1$ en promedio
Insertar	$O(\log n)$	$O(1)$ promedio; $\Theta(n)$ la que hace rehash	la misma búsqueda y un nodo nuevo; el rehash se reparte entre las inserciones que lo siguen
Borrar	$O(\log n)$	$O(1)$ promedio	se busca la clave y se desenlaza su nodo
Recorrer	$\Theta(n)$, en orden	$\Theta(n)$, sin orden	cada pareja se visita una vez; el `unordered_map` enlaza todos sus nodos en una sola lista y no pasa por las cubetas vacías
Recorrer en orden	$\Theta(n)$	$\Theta(n \log n)$	el `unordered_map` tiene que copiar las claves a otra estructura y ordenarlas
Espacio	$\Theta(n)$	$\Theta(m+n)$, con $m \geq n$	un nodo por pareja; el `unordered_map` guarda además el arreglo de cubetas

La medición, con `unordered_map` I

Pregunta

Con las mismas claves de la medición anterior, ¿dónde queda el `unordered_map`: cerca de la tabla hash del curso o cerca del `map`?

La medición, con unordered_map II

```
void medirUnordered(int n) {
  clock_t ini = clock();
  unordered_map<int, int> u;
  int i = 0;
  while (i < n) {
    u[(int) ((long long) i * 7919 % P)] = i;
    i = i + 1;
  }
  int hallados = 0;
  i = 0;
  while (i < n) {
    if (u.count((int) ((long long) i * 7919 % P)) == 1) {
      hallados = hallados + 1;
    }
    i = i + 1;
  }
  double s = (double) (clock() - ini) / CLOCKS_PER_SEC;
  printf("unordered_map, n = %d: %.3f s (%d hallados)", n, s,
  ↪  hallados);
  printf(", %d cubetas\n", (int) u.bucket_count());
}
```

La medición, con `unordered_map` III

La tabla hash y el `map` con `medir.cpp`, el `unordered_map` con `medir_hash.cpp`, las tres en la misma sesión, sin optimizaciones; cada tiempo es la mediana de tres corridas:

n	m	Tabla hash	`map`	`unordered_map`
10^5	99 991	0,005 s	0,091 s	0,033 s
10^6	999 983	0,045 s	1,018 s	0,320 s

- La tabla del curso le gana al `map` unas dieciocho veces con $n = 10^5$ y unas veintidós con $n = 10^6$.
- El `unordered_map` es unas tres veces más rápido que el `map`, y unas siete veces más lento que la tabla del curso.

La medición, con `unordered_map` IV

- La tabla del curso arranca con m cercano a n y nunca crece, y su `insertar` no revisa si la clave ya estaba. El `unordered_map` empieza con 13 cubetas, crece 17 veces hasta 1 447 153 con $n = 10^6$ y busca la clave antes de cada inserción.

Lo que puede salir mal I

Todas las claves en la misma cubeta

Si la función hash manda todas las claves a una cubeta, la tabla es una sola lista enlazada y buscar cuesta $O(n)$. El $O(1)$ es un promedio que depende de que la función reparta bien.

El orden cambia con un rehash

El recorrido sigue las cubetas, y un rehash las cambia todas. Las mismas claves pueden salir en otro orden después de una inserción más.

Un rehash invalida los iteradores

Un iterador guardado antes de un rehash ya no sirve: no se usa ni se avanza. Las referencias a los elementos sí siguen valiendo, porque los nodos no se mueven, solo se vuelven a colgar.

¿map o unordered_map? I

map

Cuando la salida va en orden de clave, cuando se pide la menor o la mayor clave, o las claves entre a y b, y cuando hace falta $O(\log n)$ también en el peor caso.

unordered_map

Cuando solo se busca, se cuenta o se suma por clave, y el orden de las claves no aparece en la salida. Es la tabla hash del curso, ya escrita, con el crecimiento incluido.

Contenido I

1 Repaso: la tabla hash con encadenamiento

2 Un problema donde importa el orden

3 El contenedor `map`

4 Lo que cuesta

5 El contenedor `unordered_map`

6 Claves compuestas en `map` y `unordered_map`

7 Para el juez

8 Ejercicios

Una persona como clave I

Problema

Una oficina guarda en qué turno llegó cada persona. La clave es la persona entera: nombre, edad y salario.

```
struct Persona {
  string nombre;   // se lee en char[101] y se copia
  int edad;
  int salario;
};
```

Pregunta

¿Compila `map<Persona, int> turno;` con solo esto? ¿Y `unordered_map<Persona, int>`?

Una persona como clave II

- No compila ninguno de los dos. El `map` compara claves con <, y `Persona` no lo trae. `unordered_map` pide una función hash y ==, y `std::hash` no sabe nada de `Persona`.
- Se le dan como el == de `Estudiante` en la sesión de iteradores o el + de `Racional`: una función `operator<` u `operator==` dentro del `struct`, que C++ llama cuando se escribe `a < b` o `a == b`.

Para el `map`: un orden I

Orden lexicográfico por campos

Primero la edad; si empatan, el salario; si también empatan, el nombre. Es el orden de un diccionario, con los campos como letras.

Primera forma: `operator<` dentro de `Persona`.

```
// edad, luego salario, luego nombre
bool operator<(const Persona &o) const {
  bool menor;
  if (edad != o.edad) {
    menor = edad < o.edad;
  } else if (salario != o.salario) {
    menor = salario < o.salario;
  } else {
    menor = nombre < o.nombre;
  }
  return menor;
}
```

Para el `map`: un orden II

Segunda forma: un `struct` comparador con `operator()`, que se pasa como tercer parámetro del `map`.

```
// El mismo orden, como un struct aparte que se pasa al map
struct CompararPersona {
  bool operator()(const Persona &a, const Persona &b) const {
    bool menor;
    if (a.edad != b.edad) {
      menor = a.edad < b.edad;
    } else if (a.salario != b.salario) {
      menor = a.salario < b.salario;
    } else {
      menor = a.nombre < b.nombre;
    }
    return menor;
  }
};
```

Para el map: un orden III

```
map<Persona, int> turno;                          // usa operator< de
↪  Persona
map<Persona, int, CompararPersona> turno2;        // usa el comparador
↪  aparte
map<Persona, int, SoloEdad> malo;                 // compara solo la
↪  edad
```

- Las dos formas dan el mismo orden y el mismo map.
- Cada comparación cuesta $\Theta(1)$ si deciden la edad o el salario, y hasta $O(L)$ si hay que comparar nombres de largo L. Las operaciones del map pasan a $O(L \log n)$.

Traza del recorrido I

Pregunta

Llegan, en este orden, `ana 30 2000`, `beto 25 3000`, `carla 30 2000`, `dario 30 1500` y `eva 25 3000`. ¿En qué orden las recorre el `map`?

Traza del recorrido II

Edad	Salario	Nombre	Llegó	Qué la ubica
25	3000	`beto`	1	la edad: 25 antes que 30
25	3000	`eva`	4	empata en edad y salario: el nombre
30	1500	`dario`	3	empata en edad: el salario
30	2000	`ana`	0	empata en edad: el salario
30	2000	`carla`	2	empata en edad y salario: el nombre

Es la salida de `persona_map.cpp` con su entrada de ejemplo: el recorrido muestra los tres desempates, y `size()` vale 5 con las dos formas.

Un comparador que solo mira la edad I

```
// Solo mira la edad: dos personas de la misma edad quedan como una
struct SoloEdad {
  bool operator()(const Persona &a, const Persona &b) const {
    return a.edad < b.edad;
  }
};
```

Pregunta

Con `map<Persona, int, SoloEdad>` y las mismas cinco personas, ¿cuánto vale `size()`?

Un comparador que solo mira la edad II

Salida de `persona_map.cpp`

```
size: 5 con operator<, 5 con CompararPersona, 2 con SoloEdad
SoloEdad: 25 3000 beto -> 4
SoloEdad: 30 2000 ana -> 3
```

Dos claves distintas, tomadas como iguales

El `map` decide que a y b son la misma clave cuando ni $a < b$ ni $b < a$. Con `SoloEdad`, dos personas de la misma edad son la misma clave: `eva` cayó sobre `beto` y le cambió el valor a 4, y `carla` y `dario` cayeron sobre `ana`. Tres personas se perdieron. Con `<=` en lugar de `<` pasa algo peor: una clave queda menor que sí misma y el `map` deja de encontrar lo que guardó.

Para el unordered_map: hash e igualdad I

La igualdad compara los tres campos:

```
// la misma persona: los tres campos iguales
bool operator==(const Persona &o) const {
  return edad == o.edad && salario == o.salario
         && nombre == o.nombre;
}
```

El hash mezcla los tres: Horner sobre el nombre, y después la edad y el salario como dos "caracteres" más.

Para el unordered_map: hash e igualdad II

```
// Horner en base 31 sobre el nombre, y despues la edad y el salario
struct HashPersona {
  unsigned long long operator()(const Persona &p) const {
    unsigned long long h = 0;
    int i = 0;
    while (i < (int) p.nombre.size()) {
      h = h * 31 + p.nombre[i];
      i = i + 1;
    }
    h = h * 31 + p.edad;
    h = h * 31 + p.salario;
    return h;
  }
};
```

Para el unordered_map: hash e igualdad III

```
unordered_map<Persona, int, HashPersona> turno;
```

Salida de persona_unordered.cpp

```
size = 5, count(carla 30 2000) = 1
count(carla 30 1500) = 0
```

- `h` no se reduce en cada paso: un `unsigned long long` que se pasa de $2^{64} - 1$ sigue la cuenta módulo 2^{64}. La reducción a la cubeta la hace el `unordered_map`.
- `HashPersona` cuesta $\Theta(L)$ y `==`, $O(L)$. Buscar cuesta $O(L)$ en promedio.

¿Cuántas chocan? I

Dos combinaciones malas para comparar:

```
// Dos combinaciones malas: solo la edad, y la edad mas el salario
struct HashEdad {
  unsigned long long operator()(const Persona &p) const {
    return p.edad;
  }
};

struct HashSuma {
  unsigned long long operator()(const Persona &p) const {
    return p.edad + p.salario;
  }
};
```

Pregunta

Entran 2000 personas: 10 nombres, edades de 20 a 39 y salarios de 1000 a 1090, de 10 en 10. ¿Cuántas cubetas usa cada función hash?

¿Cuántas chocan? II

`colisiones_persona.cpp` inserta las 2000 con cada función y recorre las cubetas con `bucket_size(b)`:

Función hash	Cubetas	Cubetas usadas	Lista más larga
solo la edad	2 357	20	100
edad + salario	2 357	110	20
los tres campos	2 357	1 487	3

- Con solo la edad hay 20 valores posibles: 100 personas por lista, y buscar recorre hasta 100 nodos.
- La suma tiene 110 valores, y además hace chocar parejas distintas: $(30, 1000)$ y $(20, 1010)$ suman 1030.
- Con los tres campos, las listas miden a lo sumo 3.

Lo que puede salir mal con el hash I

Dos claves iguales con hash distinto

Si == dice que dos claves son iguales, el hash tiene que dar el mismo valor. Si el hash usa un campo que == no mira, dos personas "iguales" caen en cubetas distintas: `count` busca en la cubeta equivocada y responde 0, y la misma clave entra dos veces.

Un hash que ignora campos

Al revés no se rompe nada: un hash que solo mira la edad da respuestas correctas, pero lentas, como en la tabla anterior.

Qué le pide cada contenedor a la clave I

	`map`	`unordered_map`
Lo que pide	un orden estricto: `operator<` o un comparador	una función hash y == (o un `struct` de igualdad)
Si falla	con un orden que no distingue campos, claves distintas se toman como iguales	con iguales de hash distinto, una clave entra dos veces
Buscar, insertar, borrar	$O(\log n)$ comparaciones; con nombres de largo L, $O(L \log n)$	un hash $\Theta(L)$ y $O(1)$ comparaciones en promedio: $O(L)$
Recorrer	en el orden del comparador, $\Theta(n)$	sin orden, $\Theta(n)$

Contenido I

1 Repaso: la tabla hash con encadenamiento

2 Un problema donde importa el orden

3 El contenedor `map`

4 Lo que cuesta

5 El contenedor `unordered_map`

6 Claves compuestas en `map` y `unordered_map`

7 Para el juez

8 Ejercicios

Para el juez: UVa 11286, Conformity I

UVa 11286, Conformity
`https://vjudge.net/problem/UVA-11286`
Envío: `https://vjudge.net/problem/UVA-11286`

Qué pide

Varios casos, terminados por una línea con 0. Cada caso trae n estudiantes, $1 \leq n \leq 10\,000$, y cada estudiante escoge cinco cursos distintos, numerados entre 100 y 499. La popularidad de una combinación es cuántos estudiantes escogieron exactamente esos cinco cursos. Se imprime cuántos estudiantes tienen una combinación de popularidad máxima.

Para el juez: UVa 11286, Conformity II

¿Entendimos el problema?

En el primer caso del enunciado, el primer y el tercer estudiante escogen 100, 101, 102, 103 y 488 en distinto orden: esa combinación tiene popularidad 2 y la respuesta es 2. En el segundo caso las tres combinaciones son distintas y cada una tiene popularidad 1. ¿Por qué la respuesta es 3 y no 1?

Las tres empatan en el máximo, y cuentan los estudiantes de todas.

Cómo atacarlo

Se ordenan los cinco cursos con `sort` y se pegan en un número de 15 cifras, tres por curso: esa es la clave de un `map<long long, int>` que cuenta. Al final, un recorrido halla la popularidad mayor y suma los estudiantes de las combinaciones que la alcanzan. Cuesta $O(n \log n)$ por caso y $\Theta(n)$ de espacio. El orden de las claves no se usa: la tabla hash también sirve, con una h para claves de 15 cifras.

Para el juez: UVa 11286, Conformity III

Las trampas

- El orden de los cursos en la línea no importa. Sin ordenar, `100 101 102 103 488` y `103 102 101 488 100` serían dos claves distintas.
- Se imprime el total de estudiantes en las combinaciones empatadas en el máximo, no cuántas combinaciones son.
- La clave no cabe en `int`: 499 499 499 499 499 pide `long long`, que se lee e imprime con `%lld`.

Para el juez: UVa 10295, Hay Points I

UVa 10295, Hay Points
`https://vjudge.net/problem/UVA-10295`
Envío: `https://vjudge.net/problem/UVA-10295`

Qué pide

Un diccionario de $m \leq 1000$ palabras, de hasta 16 letras minúsculas, cada una con un valor en dólares, un número real entre 0 y 1 000 000. Después vienen $n \leq 100$ descripciones de cargo, de una o más líneas cada una, terminadas por una línea con un punto. Por cada descripción se imprime la suma de los valores de todas las palabras que aparecen en ella; las que no están en el diccionario valen 0.

Para el juez: UVa 10295, Hay Points II

¿Entendimos el problema?

En la primera descripción del enunciado aparecen `administer` (100 000), `spending` dos veces (400 000), `money` dos veces (150 000), `responsibility` dos veces (50 000), `expertise` (100) y `skill` (50): 700 150. ¿Cuánto vale `managing`?

Cero: el diccionario tiene `manage`, y la palabra tiene que coincidir completa.

Cómo atacarlo

El diccionario va en un `map<string, double>`. Cada descripción se lee palabra por palabra con `scanf` hasta la palabra `.`, que se detecta con `strcmp`, y cada palabra se consulta con `find`. Con w palabras en una descripción, cuesta $O(w \log m)$ comparaciones de palabras.

Para el juez: UVa 10295, Hay Points III

Las trampas

- Consultar con `valor[pal]` inserta con 0 cada palabra que no está. La suma no cambia, pero el diccionario crece con cada palabra de las descripciones y las búsquedas se alargan. `find` no inserta.
- El valor es un número real: se lee como `double` con `%lf`. El ejemplo imprime enteros, y la suma se imprime con `%.0f`.
- Una descripción ocupa varias líneas: se lee por palabras, no por líneas, y termina en la palabra que es solo un punto.

Contenido I

1 Repaso: la tabla hash con encadenamiento

2 Un problema donde importa el orden

3 El contenedor `map`

4 Lo que cuesta

5 El contenedor `unordered_map`

6 Claves compuestas en `map` y `unordered_map`

7 Para el juez

8 Ejercicios

Ejercicio en parejas: a mano I

```
map<int, int> m;
m[40] = 3;
m[15] = 8;
m[72] = 1;
m[15] = m[15] - 8;
m.erase(72);
m.erase(99);
if (m[60] == 0) {
  m[33] = 2;
}
```

Las preguntas

1. ¿Qué parejas quedan, en el orden en que las recorre un iterador?
2. ¿Cuánto vale `m.size()`?
3. ¿Qué línea inserta una clave a la que nadie le asigna valor?

Ejercicio en parejas: a mano II

Lo que debe salir

1. (15, 0) (33, 2) (40, 3) (60, 0).
2. 4.
3. La condición del `if`: `m[60]` inserta (60, 0) para poder compararlo con 0. Como da 0, entra también (33, 2).

- Dejar el valor de 15 en 0 no lo saca: sigue en el `map` hasta un `erase(15)`.
- `m.erase(99)` devuelve 0 y no cambia nada.

Ejercicio en parejas: quiénes faltaron I

Se tienen los códigos de los n_a inscritos de un curso y los de los n_b que asistieron. Hay que imprimir los que faltaron.

Las preguntas

1. Con inscritos $\langle 4021, 3307, 5590, 1288, 4410, 2765 \rangle$ y asistentes $\langle 5590, 4021, 4410 \rangle$, ¿quiénes faltaron?
2. Escriban `faltaronHash` con la tabla hash de la sesión de tablas hash, en el orden de la lista de inscritos. ¿Cuánto cuesta?
3. Ahora la lista sale de menor a mayor código. Escriban `faltaronMap`. ¿Cuánto cuesta?

Ejercicio en parejas: quiénes faltaron II

```
// en el orden de la lista de inscritos: n_b inserciones y n_a
↪  busquedas, O(n_a + n_b) en promedio
void faltaronHash(int a[], int na, int b[], int nb) {
  TablaHash vinieron(11);
  int i = 0;
  while (i < nb) {
    vinieron.insertar(b[i]);
    i = i + 1;
  }
  i = 0;
  while (i < na) {
    if (!vinieron.buscar(a[i])) {
      printf(" %d", a[i]);
    }
    i = i + 1;
  }
  printf("\n");
}
```

Ejercicio en parejas: quiénes faltaron III

```
// de menor a mayor codigo: n_a + n_b operaciones de O(log n_a)
void faltaronMap(int a[], int na, int b[], int nb) {
  map<int, int> falta;
  int i = 0;
  while (i < na) {
    falta[a[i]] = 1;
    i = i + 1;
  }
  i = 0;
  while (i < nb) {
    falta.erase(b[i]);
    i = i + 1;
  }
  map<int, int>::iterator it = falta.begin();
  while (it != falta.end()) {
    printf(" %d", it->first);
    ++it;
  }
  printf("\n");
}
```

Ejercicio en parejas: quiénes faltaron IV

Lo que debe salir

1. 3307, 1288 y 2765. La tabla hash imprime `3307 1288 2765`, en el orden de los inscritos.
2. n_b inserciones y n_a búsquedas de $\Theta(1+\alpha)$: $\Theta(n_a + n_b)$ en promedio, con m proporcional a n_b.
3. El `map` imprime `1288 2765 3307`. Son n_a inserciones y n_b llamadas a `erase`, cada una $O(\log n_a)$: $O((n_a + n_b)\log n_a)$.

`erase` de un asistente que no estaba inscrito devuelve 0 y no hace nada: no hace falta preguntar antes.

Ejercicio en parejas: la moda I

La moda de un arreglo es el valor que más se repite. Si hay empate, se responde el menor.

Las preguntas

1. Con $a = \langle 5, 2, 8, 2, 5, 9, 5, 2 \rangle$, ¿qué responde?
2. Escriban `moda` con un `map<int, int>`. ¿Por qué basta comparar con > estricto para resolver el empate?
3. ¿Qué cambia si se cuenta con la tabla hash?

Ejercicio en parejas: la moda II

```
int moda(int a[], int n) {
  map<int, int> veces;
  int i = 0;
  while (i < n) {
    veces[a[i]] = veces[a[i]] + 1;
    i = i + 1;
  }
  int mejor = 0;
  int cuantas = 0;
  map<int, int>::iterator it = veces.begin();
  while (it != veces.end()) {
    if (it->second > cuantas) {     // con > estricto, en un empate
    ↪  se queda la menor
      mejor = it->first;
      cuantas = it->second;
    }
    ++it;
  }
  return mejor;
}
```

Ejercicio en parejas: la moda III

Lo que debe salir

1. 2. El 2 y el 5 aparecen tres veces cada uno, y 2 es el menor.
2. El recorrido va de menor a mayor clave: la primera clave que alcanza la cuenta mayor es la menor de las empatadas, y con > estricto las siguientes no la reemplazan.
3. La tabla recorre por casillas, no por clave. En el empate hay que comparar también las claves, y cada nodo tiene que guardar su cuenta. Cuesta $\Theta(n)$ en promedio, frente a $O(n \log n)$ del `map`.

¿Tabla hash o `map`? I

Para cada situación, ¿qué estructura escogen, y por qué?

1. Un corrector revisa un millón de palabras contra un diccionario de cien mil y solo dice si cada una está.
2. Una agenda imprime los contactos en orden alfabético después de cada cambio.
3. Un sistema de turnos pide, en cualquier momento, el código más pequeño registrado.
4. Se cuenta cuántas veces aparece cada número de una lista de un millón para imprimir los cinco más frecuentes.

¿Tabla hash o `map`? II

Respuestas

1. Tabla hash: un millón de búsquedas y ningún orden en la salida.
2. `map`: el recorrido ya sale en orden, $\Theta(n)$ por impresión, sin ordenar cada vez.
3. `map`: `begin()` da la menor clave en $\Theta(1)$.
4. Tabla hash. La salida se ordena por frecuencia, no por clave, así que el orden del `map` no ahorra nada.

Ejercicio en parejas: otro orden para las personas I

Enunciado

Escriban un comparador `PorSalario` para `map<Persona, int, PorSalario>` que ponga primero el salario mayor y, si empatan, ordene por nombre. Con las cinco personas de la traza, ¿en qué orden salen y cuánto vale `size()`? ¿Qué pasa si llega `ana 41 2000`?

Ejercicio en parejas: otro orden para las personas II

```
struct PorSalario {
  bool operator()(const Persona &a, const Persona &b) const {
    bool menor;
    if (a.salario != b.salario) {
      menor = a.salario > b.salario;   // el salario mayor va primero
    } else {
      menor = a.nombre < b.nombre;
    }
    return menor;
  }
};
```

Ejercicio en parejas: otro orden para las personas III

Salida de `persona_salario.cpp`

```
3000 beto, 3000 eva, 2000 ana, 2000 carla, 1500 dario;
size = 5.
```

`PorSalario` no mira la edad: `ana 41 2000` empata en salario y nombre con `ana 30 2000`, se toma como la misma clave y `size()` sigue en 5. $O(L)$ por comparación.

Propuestos para la casa I

1. **Entradas y salidas.** Cada registro del inventario trae un tipo, 1 para entrada y 2 para salida, un código y una cantidad. Un código cuyo total llega a 0 desaparece. Al final se imprimen los totales en orden de código.
2. **Distintos en orden.** Imprimir los valores distintos de un arreglo de menor a mayor, una vez con un `map` y otra con la tabla hash y `sort`. ¿Cuánto cuesta cada versión?
3. **Anagramas.** Decidir si dos palabras usan las mismas letras el mismo número de veces, con un `map<char, int>`. Con solo letras minúsculas, ¿qué estructura de la sesión de tablas directas sirve mejor?
4. **Hay Points con la tabla hash.** Resolver UVa 10295 con una tabla hash de palabras cuyo nodo guarda la palabra y su valor.
5. **Promedio por estudiante.** Cada registro trae un código y una nota en décimas. Imprimir, en orden de código, el promedio de cada estudiante.

Pistas para los propuestos I

1. Se suma o se resta con `[]`; si el total queda en 0, se llama a `erase`. $O(\log n)$ por registro.
2. Con el `map`, se insertan los valores y se recorre: $O(n \log n)$. Con la tabla, se inserta cada valor que no estaba, se copian los distintos a un arreglo y se ordena: $\Theta(n)$ en promedio más $\Theta(d \log d)$ para los d distintos.
3. Se suma uno por cada letra de la primera y se resta uno por cada letra de la segunda; son anagramas si todos los valores terminan en 0. Con 26 letras, una tabla directa de 26 casillas hace cada paso en $\Theta(1)$.
4. La casilla sale del polinomio en base 31 y la comparación de la lista pasa a `strcmp`. Las palabras que no están no se insertan.
5. Dos `map<int, int>`, uno con la suma y otro con la cantidad de notas, o uno solo cuyo valor sea un `struct` con las dos.

Reto de cierre I

Claves en un rango

Un `map<int, int>` tiene n claves. Dados $a \leq b$, hay que imprimir las claves que están entre a y b. Recorrer todo el `map` cuesta $\Theta(n)$ aunque en el rango haya dos claves. ¿Se puede pagar algo cercano a $\log n$ más lo que se imprime? ¿Qué operación le faltaría al contrato para lograrlo?

Ejercicio de cierre: la primera palabra sin repetir I

Problema

Se leen palabras hasta el fin de la entrada. Hay que imprimir la primera, en el orden de llegada, que aparece una sola vez, o `ninguna`.

Ejemplo

Con `sol mar luna sol mar rio`, la respuesta es `luna`: `rio` también aparece una vez, pero llega después.

Pregunta

El `unordered_map` cuenta en $O(1)$ promedio por palabra, pero su recorrido no respeta el orden de llegada. ¿Cómo se recupera ese orden, y cuánto cuesta todo?

Ejercicio de cierre: respuesta I

```cpp
int main() {
  vector<string> texto;                    // las palabras en el orden de
  ↪  llegada
  unordered_map<string, int> veces;
  char pal[101];
  while (scanf("%s", pal) == 1) {
    string clave = pal;
    texto.push_back(clave);
    veces[clave] = veces[clave] + 1;
  }
  int i = 0;
  while (i < (int) texto.size() && veces[texto[i]] != 1) {
    i = i + 1;
  }
  if (i < (int) texto.size()) {
    printf("%s\n", texto[i].c_str());
  } else {
    printf("ninguna\n");
  }
  return 0;
}
```

Ejercicio de cierre: respuesta II

- El orden de llegada se guarda aparte, en un `vector`. La segunda pasada recorre el `vector`, no el mapa.
- Salida con el ejemplo: `luna`. Con `sol mar sol mar`: `ninguna`.
- Costo: n palabras de largo a lo sumo L, dos pasadas con un hash $\Theta(L)$ por palabra: $O(nL)$ en promedio. Espacio: $\Theta(nL)$ para el `vector` y el mapa. Con un `map` serían $O(nL \log n)$.

Resumen I

- La tabla hash recorre por casillas: imprimir en orden de clave le exige ordenar.
- Un `map<K, V>` guarda parejas con claves distintas en orden creciente: `m[k]` lee o asigna, `count` y `find` consultan, `erase` saca, y un iterador recorre de la menor clave a la mayor.
- `m[k]` inserta $(k, 0)$ si `k` no estaba: sirve para contar y estorba para preguntar.
- Una clave de texto va como `string`; un `char *` se compara por dirección.
- Buscar, insertar y eliminar cuestan $O(\log n)$ también en el peor caso; recorrer, $\Theta(n)$. Con un millón de claves la tabla hash fue unas diez veces más rápida, y el `map` se escoge cuando el orden de las claves está en la salida.
- `unordered_map` es una tabla hash con encadenamiento: la misma interfaz del `map`, $O(1)$ en promedio, rehash cuando α pasa de 1 y un recorrido que no sale en orden.

Resumen II

- Una función hash es determinista, reparte parejo y es rápida: división con m primo, multiplicación con cualquier m, Horner para cadenas. En g++, `std::hash<int>` devuelve el mismo entero.
- Una clave compuesta necesita un orden estricto en el `map`, y un hash que mezcle todos los campos más == en el `unordered_map`.

Lo que sigue

Los árboles: la estructura con la que la biblioteca guarda el `map`, y de donde sale su $O(\log n)$.

Referencias I



T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein. *Introduction to Algorithms*, 4.ª ed. MIT Press, 2022. Capítulo 11, tablas hash (§11.2, encadenamiento y su análisis); capítulo 12, árboles binarios de búsqueda; capítulo 13, árboles rojinegros, donde buscar, insertar y eliminar cuestan $O(\log n)$.



cppreference.com. *std::map*: `https://en.cppreference.com/w/cpp/container/map`. Contrato y complejidad de cada operación.



cppreference.com. *std::unordered_map*: `https://en.cppreference.com/w/cpp/container/unordered_map`. Cubetas, factor de carga, rehash e invalidación de iteradores.



UVa Online Judge. Problema 11286, *Conformity*: `https://vjudge.net/problem/UVA-11286`.



UVa Online Judge. Problema 10295, *Hay Points*: `https://vjudge.net/problem/UVA-10295`.